perf_analyzer 使用说明书

Oct 10, 2026 · @john.he

1. 简介

`perf_analyzer` 是一个运行在主机（Ubuntu）上的命令行工具，读取 perf 采集并解析好符号的数据，自动输出一份中文分析报告：数据是否可信、CPU 花在哪些线程和函数上、命中了哪些常见性能问题模式、优先优化什么。它把人工看火焰图的分析思路固化成了规则，适合对 E3 板子上的 `ApplicationServices` 等程序反复做性能分析。

项目	说明
运行环境	主机 Linux（已在 Ubuntu 22.04 验证），单文件 C++17，无第三方依赖
输入	`perf script` 的文本输出（推荐），或 `stackcollapse-perf.pl` 生成的折叠栈，格式自动识别
输出	Markdown 格式的分析报告；可选输出折叠栈，直接交给 `flamegraph.pl` 生成火焰图
配套文件	`perf_analyzer.cpp`（源码）、`Makefile`（编译和一键生成报告）

报告包含七部分：数据质量检查、整体概况、线程分布与最热调用路径、自身/累计耗时排行、规则检测结果、优化优先级、说明。

2. 安装与编译

环境要求：主机上有 `g++`（支持 C++17）、`make`、`perf`；生成火焰图还需要 FlameGraph 脚本（`git clone https://github.com/brendangregg/FlameGraph.git`）。

源码结构（全部放在同一目录，例如 `/home/hejr/test/perfAnalyzer`）：

文件	内容
`main.cpp`	程序入口，串联各模块

文件	内容
`options.h/.cpp`	命令行选项和帮助信息
`input_parser.h/.cpp`	`InputParser` 类：格式识别，解析 perf script 和折叠栈
`dataset.h/.cpp`	`Interner`（函数名字符串池）、`Stack`、`Dataset`（含输出折叠栈）
`symbol_rules.h/.cpp`	函数分类规则：内核/库/业务、系统调用入口、栈底异常帧、可疑 libc 函数
`analyzer.h/.cpp`	`Analyzer` 类：统计、报告输出、规则检测
`string_utils.h/.cpp`	字符串和格式化工具
`Makefile`	编译、安装、一键生成报告

旧的单文件 `perf_analyzer.cpp` 不再使用，请从目录中删除。

```
cd /home/hejr/test/perfAnalyzer
make -j4        # 编译，生成 ./perf_analyzer（指向 build/perf_analyzer）
make debug      # 调试版本：-O0 -g，带地址和未定义行为检查
make install    # 安装到 /usr/local/bin（需要 root）
make clean      # 清理 build/ 和 perf_analyzer，不会删除源码
```

Makefile 按文件分别编译并自动跟踪头文件依赖，修改某个 `.h` 后只重新编译相关的 `.cpp`。

注意

- Makefile 不继承环境变量中的 `CROSS_COMPILE`，即使 shell 里 source 过 SDK 环境脚本，`make` 仍使用主机 `g++`。
- 需要放到 E3 板子上运行时，显式交叉编译：`make clean && make CROSS_COMPILE=aarch64-linux-gnu- LDFLAGS=-static`，再用 `aarch64-linux-gnu-strip build/perf_analyzer` 精简后拷贝。板子上没有用户态符号时，只能得到内核函数和库导出函数相关的结论。
- 用 root 执行时 `~` 和 `$HOME` 指向 `/root`，hejr 用户目录下的文件要写完整路径（如 `/home/hejr/perfwork`）。

3. 快速开始

三步得到报告和火焰图。前提：板子上运行的程序与主机符号目录（`/home/hejr/symfs_e3`）是同一次编译的产物，已用 md5 核对过。

1. **板子上采集**（在业务繁忙或问题复现时）

```
echo 0  > /proc/sys/kernel/kptr_restrict
echo -1 > /proc/sys/kernel/perf_event_paranoid
cd /tmp
perf record -e cpu-clock -F 199 -m 64 --call-graph fp --proc-map-timeout
10000 \
    -p $(pidof ApplicationServices) -o perf.data -- sleep 120
cat /proc/kallsyms > kallsyms
```

2. **拷贝到主机**

```
scp root@10.166.64.107:/tmp/{perf.data,kallsyms} /home/hejr/perfwork/
```

3. **一键生成报告和火焰图**

```
make -C /home/hejr/test/perfAnalyzer report \
    DATA=/home/hejr/perfwork/perf.data \
    KALLSYMS=/home/hejr/perfwork/kallsyms \
    SYMFS=/home/hejr/symfs_e3 \
    FLAMEGRAPH=/home/hejr/FlameGraph \
    DURATION=120
```

结果在 `/home/hejr/test/perfAnalyzer/report/`：`report.md` 分析报告、`flame.svg` 火焰图，另有中间文件 `out.perf`、`out.folded` 和 perf 解析日志 `perf_script.log`。`DURATION` 填第 1 步 `sleep` 的秒数。

4. 详细用法

4.1 板子端采集要点

- 用 `cpu-clock` 软件事件，比硬件 `cycles` 可靠；`-F 199` 配合 120 秒，样本数能到几千个。只看用户态时改成 `cpu-clock:u`。
- `--proc-map-timeout 10000`：用 `-p` 附加到运行中的进程时给读取内存映射留足时间，否则样本会全部归到 `[JIT]`。

- 导出 `kallsyms` 前必须设置 `kptr_restrict=0`，且驱动模块仍处于加载状态，否则内核函数显示为 `[unknown]`。
- 板子上的 perf 读不到 build-id，用 `md5sum` 核对程序版本：主机 `rootfs_arm64-short/rootfs_partition/` 下的文件应与板子上的完全一致。

4.2 主机端手动分析

```
cd /home/hejr/perfwork
perf script --force -i perf.data --symfs=/home/hejr/symfs_e3 --
kallsyms=kallsyms > out.perf
/home/hejr/test/perfAnalyzer/perf_analyzer out.perf --duration 120 >
report.md
```

`--force` 在 root 读取 hejr 用户的 `perf.data` 时需要；用 hejr 用户执行可以省略。只想看某个线程：加 `--comm DataServices`。

4.3 命令行参数

参数	说明	默认值
`<输入文件>`	`perf script` 输出或折叠栈文件	必填
`--format auto|script|folded`	输入格式	`auto`（自动识别）
`--duration <秒>`	采集时长，用于计算平均 CPU 占用	不计算
`--freq <Hz>`	采样频率，输入中没有 period 信息时用于估算 CPU 时间	—
`--top <N>`	热点排行条数	15
`--min-pct <百分比>`	规则检测的最小占比阈值	1.0
`--comm <线程名>`	只分析指定线程	全部
`--keep-root-anomalies`	保留栈底的 `[perf-N.map]` / `[JIT]` / `[unknown]` 帧	自动去除
`--folded-out <文件>`	同时输出折叠栈，可直接交给 `flamegraph.pl`	不输出
`--mark-kernel`	折叠栈中内核帧加 `_[k]` 后缀（配合 `flamegraph.pl --color=java` 区分颜色）	不加

4.4 Makefile 目标与变量

目标	作用
`make` / `make release`	编译
`make debug`	编译调试版本
`make report`	perf script 解析 → 分析报告 → 火焰图
`make install` / `make uninstall`	安装到 / 卸载自 `$(PREFIX)/bin`
`make clean`	清理编译产物
`make help`	显示帮助

变量（用于 `make report`）	说明	默认值
`DATA`	perf.data 路径	`perf.data`
`KALLSYMS`	板子导出的 kallsyms 路径	`kallsyms`
`SYMFS`	符号目录	`$(HOME)/symfs_e3`
`DURATION`	采集秒数	0（不计算 CPU 占用）
`FLAMEGRAPH`	FlameGraph 脚本目录	`$(HOME)/FlameGraph`
`TITLE`	火焰图标题	`ApplicationServices`
`OUT_DIR`	输出目录	`report`
`PERF`	perf 命令	`perf`
`PERF_SCRIPT_FLAGS`	传给 perf script 的额外参数	`--force`
`PREFIX`	安装路径前缀	`/usr/local`

5. 报告解读

先看第一部分判断数据是否可信，再看第五、六部分决定优化方向；所有占比都相对于全部样本。

章节	内容	怎么看
一、数据质量检查	样本数、未解析符号比例、libc 版本不匹配、栈底异常帧、回溯断裂	出现"libc 版本不一致"或样本数很少时，后面的结论要打折扣，先按提示修正
二、整体概况	样本数、CPU 时间、平均 CPU 占用、内核和系统调用占比、系统调用分布	平均占用很低时，热点只是相对比例，优化的绝对收益有限
三、线程分布	各线程占比、每个线程累计占比最高的业务函数、主要线程的最热调用路径	先找最耗 CPU 的线程，再顺着调用路径往下看
四、热点排行	自身耗时（采样点直接落在函数里）和累计耗时（含子调用，只列业务函数）	自身耗时高说明函数本身慢；累计高但自身低说明耗时在它调用的函数里
五、规则检测结果	命中的问题模式，附调用者、所在线程和建议	调用者是具体的优化落点

章节	内容	怎么看
六、优化优先级	按"占比 ÷ 改动难度"排序	从上往下依次处理
七、说明	统计口径和局限	—

规则检测说明

规则	识别依据	含义与建议
频繁调用 gettid	`gettid`	每次都是系统调用；在线程入口获取一次并缓存
频繁获取系统时间	`clock_gettime`、`gettimeofday`	64 位内核上的 32 位程序常走系统调用；一个周期只取一次
64 位软件除法	`__udivmoddi4`、`__aeabi_uldivmod` 等	32 位 ARM 没有 64 位除法指令；常见于时间换算，减少除法次数
疑似反复排序	名字含 compare/cmp 的业务函数，且由库函数回调	每周期对整个列表 qsort；改用最小堆或有序插入
字符串键 map 查找	字符串键的 `_Rb_tree` / `_Hashtable`，以及 `memcmp`、`strlen`、`std::string` 构造	按名字查属性；初始化时解析成索引或指针缓存
等待类调用占用 CPU	`select`、`poll`、`epoll_wait`、`nanosleep` 等	线程被频繁唤醒，检查超时是否过短
内存分配频繁	`malloc`、`free`、`operator new` 等	减少临时对象和拷贝，用 reserve、复用、move
锁和信号量	`sem_trywait`、`pthread_mutex_*`、`PosixSemTake` 等	检查轮询式加锁、锁粒度
JSON 解析	`Json::Reader::*`、`nlohmann::*` 等	避免周期性重复解析，解析一次后缓存
系统调用整体占比偏高	系统调用中的样本超过 20%	汇总项，处理上面几项后会下降

规则阈值由 `--min-pct` 控制（默认 1%），样本较少时可适当调高，避免噪声。

6. 常见问题、扩展与局限

6.1 常见问题

现象	原因	处理
`make` 调用了 `arm-buildroot-linux-gnueabihf-g++` 并报找不到	使用了继承环境变量的旧版 Makefile	换用新版 Makefile，或执行 `make CROSS_COMPILE=`
`perf.data not owned by current user or root`	root 读取 hejr 用户的文件	切换到 hejr 用户，或加 `--force`（`make report` 已默认带上）
`没有解析到任何样本`	输入不是 perf script 输出或折叠栈，或 `--comm` 线程名不对	检查输入文件开头几行和线程名
报告提示 libc 版本不一致	symfs 中的 libc 与板子不同	从板子拷贝 `/lib/arm-linux-gnueabihf/libc-2.30.so` 放到 symfs 相同路径
未解析比例高，只显示 `[libc-2.30.so]`	库被 strip，内部 static 函数无法还原	正常现象；导出函数（gettid、malloc 等）仍能显示
栈底大量 `[perf-N.map]`	回溯越过栈底落到可执行匿名内存	工具已自动去除，不影响结论
业务调用链明显不合理	符号目录与板子程序版本不一致	md5 核对后重建 symfs，重新解析
样本数很少	频率低、时长短或程序空闲	`-F 199`、延长采集，在业务繁忙时采集
root 下路径找不到	`$HOME` 是 `/root`	写完整路径 `/home/hejr/...`

6.2 扩展规则

想要修改的内容	文件与位置
增加或调整问题检测规则	`analyzer.cpp` 中的 `Analyzer::detect()`

想要修改的内容	文件与位置
识别新的库函数、第三方库前缀	`symbol_rules.cpp` 中的 `kLibcNames` 和 `isLibName()`
识别新的内核函数前缀（折叠栈输入）	`symbol_rules.cpp` 中的 `looksLikeKernel()`
增加命令行选项	`options.h` 的 `Options` 和 `options.cpp` 的 `parseOptions()`
报告章节的格式	`analyzer.cpp` 中各 `print*()` 函数

每条检测规则是 `detect()` 中一段独立代码。例如增加"日志输出过多"的检测：

```
// 在 Analyzer::detect() 中、排序 findings 之前加入
{
    set<string> t = {"vfprintf", "__vfprintf_internal", "fwrite", "syslog",
"LogWrite"};
    double w = inclOf(t);              // 包含任一目标函数的样本数
    if (over(w)) {                     // 超过 --min-pct 阈值
        Finding f;
        f.title = "日志输出占用 CPU";
        f.weight = w;
        addCallerDetails(f, t);        // 自动列出业务调用者和所在线程
        f.advice = "降低周期性路径上的日志级别，或改为异步写日志。";
        f.effort = "小";               // 很小 / 小 / 中，影响优先级排序
        findings.push_back(f);
    }
}
```

改完执行 `make` 即可，只会重新编译 `analyzer.cpp`。可复用的辅助函数都是 `Analyzer` 的成员（声明在 `analyzer.h`）：`inclOf()` 统计占比，`appCallers()` 找业务调用者，`byThread()` 按线程统计，`addCallerDetails()` 一次性加入调用者和线程信息。

6.3 局限

- 规则是基于常见模式的启发式判断，不能替代看火焰图，结论需要结合业务代码确认。
- 工具无法判断业务调用链是否合理，符号版本是否匹配仍需人工用 md5 核对。
- CPU 火焰图只反映占用 CPU 的时间，阻塞、等锁、等 IO 的时间不会体现。
- 输入为折叠栈时，内核函数按名字启发式识别；用 perf script 输出作为输入更准确。
- 不同规则之间可能重叠（例如 gettid 也计入系统调用整体占比），占比不能直接相加。